

PERSONAL DIARY APPLICATION

C++ PROJECT

Features:

- **Write Diary Entry**
- **View Entries**
- **Search by Date**
- **Mood Statistics**

CODE:

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <vector>
```

```
#include <ctime>
```

```
#include <sstream>
```

```
#include <iomanip>
```

```
#include <algorithm>
```

```
#include <limits>
```

```
#include <map>
```

```
//
```

```
// DiaryEntry
```

```
//
```

```
struct DiaryEntry {
```

```
    std::string date;
```

```
    std::string time;
```

```
    std::string title;
```

```
    std::string content;
```

```
    std::string mood;
```

```
};
```

```
//
```

```
// Get Date & Time
```

```
//
```

```
std::pair<std::string, std::string> getCurrentDateTime() {
```

```
    std::time_t t = std::time(nullptr);
```

```
    std::tm* tm_info = std::localtime(&t);
```

```
    std::ostringstream date_ss, time_ss;
```

```
    date_ss << std::put_time(tm_info, "%Y-%m-%d");
```

```
    time_ss << std::put_time(tm_info, "%H:%M:%S");
```

```
    return { date_ss.str(), time_ss.str() };
```

```
}
```

```
//
```

```
// UI Helpers
```

```
//
```

```
void clearScreen() {
```

```
#ifdef _WIN32
```

```
    system("cls");
```

```

#else
    system("clear");
#endif
}

void printSeparator(char c = '-', int width = 60) {
    std::cout << std::string(width, c) << "\n";
}

void printHeader(const std::string& title) {
    clearScreen();
    printSeparator('=');
    std::cout << "  PERSONAL DIARY APPLICATION\n";
    printSeparator('=');
    std::cout << "  " << title << "\n";
    printSeparator('-');
    std::cout << "\n";
}

//
//
// File I/O
//
//
const std::string DIARY_FILE = "diary_entries.txt";

```

```
const std::string DELIMITER = "===ENTRY_END===";
```

```
void saveEntry(const DiaryEntry& entry) {  
    std::ofstream file(DIARY_FILE, std::ios::app);  
    if (!file) {  
        std::cerr << "Error opening file.\n";  
        return;  
    }
```

```
    file << "DATE:" << entry.date << "\n"  
        << "TIME:" << entry.time << "\n"  
        << "TITLE:" << entry.title << "\n"  
        << "MOOD:" << entry.mood << "\n"  
        << "CONTENT:" << entry.content << "\n"  
        << DELIMITER << "\n";  
}
```

```
// Safe prefix checker
```

```
bool startsWith(const std::string& line, const std::string& prefix) {  
    return line.size() >= prefix.size() &&  
        line.compare(0, prefix.size(), prefix) == 0;  
}
```

```
std::vector<DiaryEntry> loadEntries() {  
    std::vector<DiaryEntry> entries;
```

```

std::ifstream file(DIARY_FILE);
if (!file) return entries;

DiaryEntry current;
std::string line;

while (std::getline(file, line)) {
    if (line == DELIMITER) {
        entries.push_back(current);
        current = DiaryEntry{};
    }
    else if (startsWith(line, "DATE:"))
        current.date = line.substr(5);
    else if (startsWith(line, "TIME:"))
        current.time = line.substr(5);
    else if (startsWith(line, "TITLE:"))
        current.title = line.substr(6);
    else if (startsWith(line, "MOOD:"))
        current.mood = line.substr(5);
    else if (startsWith(line, "CONTENT:"))
        current.content = line.substr(8);
}

return entries;
}

```

```
//
```

```
// Write Entry
```

```
//
```

```
void writeNewEntry() {
```

```
    printHeader("Write New Entry");
```

```
    DiaryEntry entry;
```

```
    auto [date, time] = getCurrentDateTime();
```

```
    entry.date = date;
```

```
    entry.time = time;
```

```
    std::cout << "Date: " << date << "\n";
```

```
    std::cout << "Time: " << time << "\n\n";
```

```
    std::cout << "Title: ";
```

```
    std::getline(std::cin, entry.title);
```

```
    std::cout << "Mood (1-Happy 2-Sad 3-Neutral 4-Excited 5-Stressed): ";
```

```
    int choice;
```

```
    std::cin >> choice;
```

```
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
```

```

const std::string moods[] = {"Happy","Sad","Neutral","Excited","Stressed"};

if (choice >= 1 && choice <= 5)
    entry.mood = moods[choice - 1];
else
    entry.mood = "Neutral";

std::cout << "\nWrite your entry (empty line to finish):\n";
std::string line;
while (true) {
    std::getline(std::cin, line);
    if (line.empty()) break;
    entry.content += line + " ";
}

saveEntry(entry);

std::cout << "\nSaved successfully!\n";
std::cin.get();
}

//
//
// View Entries
//
//

void viewAllEntries() {

```



```

printHeader("All Entries");

auto entries = loadEntries();

if (entries.empty()) {
    std::cout << "No entries found.\n";
    std::cin.get();
    return;
}

for (size_t i = 0; i < entries.size(); i++) {
    std::cout << i+1 << ". " << entries[i].date
        << " | " << entries[i].title
        << " (" << entries[i].mood << ")\n";
}

std::cout << "\nSelect entry (0 to exit): ";

int n;

std::cin >> n;

std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

if (n > 0 && n <= entries.size()) {
    auto& e = entries[n-1];
    printHeader("Entry");

    std::cout << "Date: " << e.date << "\n"

```

```

        << "Time: " << e.time << "\n"
        << "Title: " << e.title << "\n"
        << "Mood: " << e.mood << "\n\n"
        << e.content << "\n";
    }

    std::cin.get();
}

//


---




---


// Search
//


---




---



void searchByDate() {
    printHeader("Search");

    std::string d;
    std::cout << "Enter date (YYYY-MM-DD): ";
    std::getline(std::cin, d);

    auto entries = loadEntries();
    bool found = false;

    for (auto& e : entries) {

```

```
        if (e.date == d) {  
            std::cout << "\n" << e.title << " | " << e.mood << "\n"  
                << e.content << "\n";  
            printSeparator();  
            found = true;  
        }  
    }  
  
    if (!found) std::cout << "No entries found.\n";  
    std::cin.get();  
}
```

```
//
```

```
// Stats
```

```
//
```

```
void viewMoodStats() {  
    printHeader("Mood Stats");  
  
    auto entries = loadEntries();  
    if (entries.empty()) {  
        std::cout << "No data.\n";  
        std::cin.get();  
        return;  
    }  
}
```

```
}
```

```
std::map<std::string,int> m;
```

```
for (auto& e : entries)
```

```
    m[e.mood]++;
```

```
for (auto& p : m) {
```

```
    std::cout << p.first << " : "
```

```
        << std::string(p.second, '*')
```

```
        << " (" << p.second << ")\n";
```

```
}
```

```
std::cin.get();
```

```
}
```

```
//
```

```
// Main
```

```
//
```

```
int main() {
```

```
    while (true) {
```

```
        printHeader("MAIN MENU");
```

```

std::cout << "1. Write\n"
    << "2. View\n"
    << "3. Search\n"
    << "4. Stats\n"
    << "5. Exit\n"
    << "Choice: ";

int ch;

std::cin >> ch;

std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

switch (ch) {
    case 1: writeNewEntry(); break;
    case 2: viewAllEntries(); break;
    case 3: searchByDate(); break;
    case 4: viewMoodStats(); break;
    case 5: return 0;
    default: std::cout << "Invalid\n";
}
}
}

```

Sample Output:

PERSONAL DIARY APPLICATION

MAIN MENU

Today: 2026-04-13 | 10:30:45

- 1. Write New Entry**
- 2. View All Entries**
- 3. Search by Date**
- 4. Mood Statistics**
- 5. Exit**

Your choice: 1

PERSONAL DIARY APPLICATION

Write New Entry

Date: 2026-04-13

Time: 10:31:02

Title: First Day Using Diary

Mood (1-Happy 2-Sad 3-Neutral 4-Excited 5-Stressed): 1

Write your entry (empty line to finish):

Today I created my diary project.

It was a great learning experience.

Saved successfully!

PERSONAL DIARY APPLICATION

MAIN MENU

Today: 2026-04-13 | 10:32:10

- 1. Write New Entry**
- 2. View All Entries**
- 3. Search by Date**
- 4. Mood Statistics**
- 5. Exit**

Your choice: 2

PERSONAL DIARY APPLICATION

All Entries

1. 2026-04-13 | First Day Using Diary (Happy)

Select entry (0 to exit): 1

PERSONAL DIARY APPLICATION

Entry

Date: 2026-04-13

Time: 10:31:02

Title: First Day Using Diary

Mood: Happy

Today I created my diary project. It was a great learning experience.

PERSONAL DIARY APPLICATION

MAIN MENU

Today: 2026-04-13 | 10:33:05

1. Write New Entry

- 2. View All Entries**
- 3. Search by Date**
- 4. Mood Statistics**
- 5. Exit**

Your choice: 3

PERSONAL DIARY APPLICATION

Search

Enter date (YYYY-MM-DD): 2026-04-13

First Day Using Diary | Happy

Today I created my diary project. It was a great learning experience.

PERSONAL DIARY APPLICATION

MAIN MENU

Today: 2026-04-13 | 10:33:50

1. Write New Entry

- 2. View All Entries
- 3. Search by Date
- 4. Mood Statistics
- 5. Exit

Your choice: 4

PERSONAL DIARY APPLICATION

Mood Stats

Happy : * (1)

Total entries: 1

PERSONAL DIARY APPLICATION

MAIN MENU

Today: 2026-04-13 | 10:34:20

- 1. Write New Entry
- 2. View All Entries
- 3. Search by Date

4. Mood Statistics

5. Exit

Your choice: 5

Goodbye! Keep writing!

